# Programación Segura

## Problemas comunes de seguridad

**Docente:** Emilio Antonio Alarcón Santos
**Contacto:** emalarcon.uv.mx

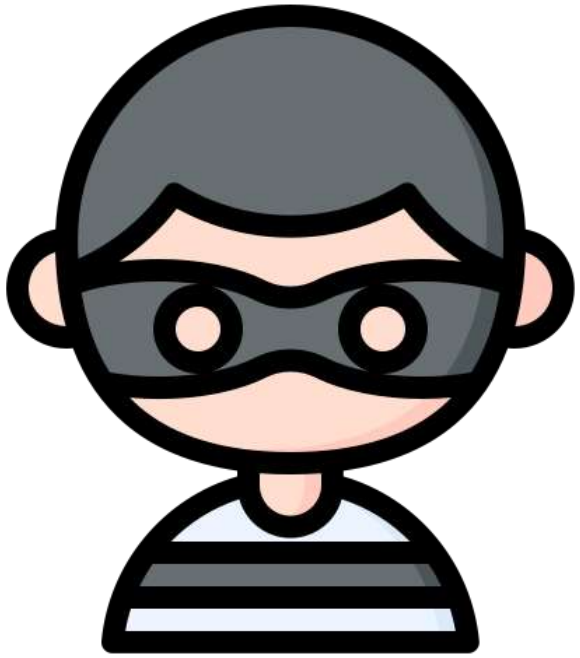


# Manejo de entradas

# Introducción

- Las entradas de un programa son probablemente el punto más critico de seguridad que le corresponde a un programador manejar

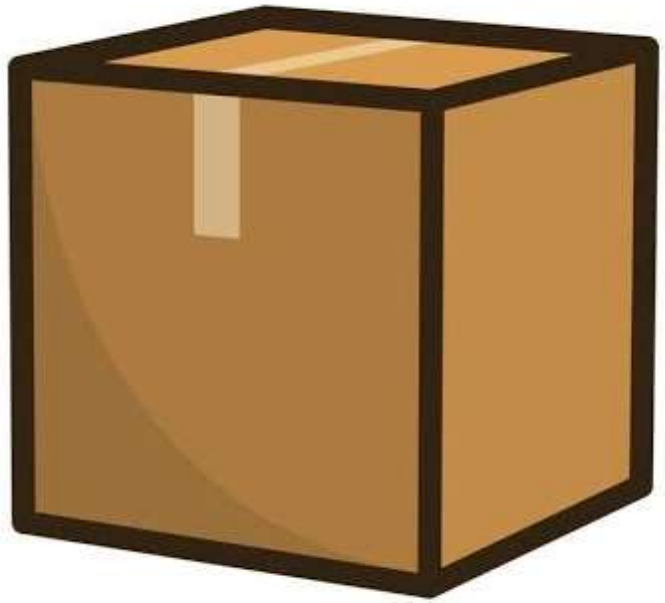






# Introducción

- Es en las entradas donde los atacantes pueden explotar vulnerabilidades de inyección, romper por fuerza bruta claves de acceso, realizar fuzzing para localizar posibles errores en el manejo de memoria, abusar de un API, etc.








# ¿Que son las entradas?



- Técnicamente las entradas son cualquier cosa que recibe alguna rutina para ser procesadas
- En seguridad especialmente nos interesan las entradas externas
- Las entradas externas pueden ser dadas por usuarios o bien por otros sistemas
- Nunca hay que confiar en una entrada externa
- Por otro lado las entradas internas no suelen ser un riesgo para el cual haya que tenerse un énfasis de seguridad especial

# Protegerse de entradas perniciosas



- Siempre valida que la entrada esté en algún formato esperado (usa expresiones regulares)
- Valida la longitud de la entrada, sobre todo las cadenas muy largas pueden ser parte de un ataque
- Si la entrada será utilizada de forma directa o indirecta como parte de un comando dentro de tu sistema (como por ejemplo un query sql), sanitiza la entrada

# Protegerse de entradas perniciosas



- En el caso de aplicaciones Web, siempre valida entradas del lado del cliente y del servidor, un atacante podría desactivar javascript o manipular el HTML o bien utilizar un bot para hacer un ataque que se salte validaciones del lado del cliente
- Limita el número de intentos de un cliente para ingresar un dato, esto mitigará en gran medida algún ataque basado en fuerza bruta o fuzzing

# Actividad



- Realiza una lectura de los siguientes tutoriales sobre uso de expresiones regulares en Java:
  - http://tutorials.jenkov.com/java-regex/pattern.html
  - http://tutorials.jenkov.com/java-regex/matcher.html
- Crea dos programas de línea de comandos en Java para resolver validar los siguientes tipos de entradas:
  - **CURP**
  - **correo electrónico**
- Los programas simplemente reciben una entrada de usuario y devuelven true o false dependiendo si la cadena se apega al formato dado

# Validación de cadenas

- Revisa que la cadena se adapta a una longitud mínima y máxima para mitigar posibles ataques de desbordamiento de buffer
- Ten especial cuidado de que una cadena no rebase el tamaño del campo establecido en la base de datos para almacenarlo
- Revisa que la cadena no contenga caracteres especiales como;
  - “ “ ” ” ; ``

# Validación de cadenas



- Si es necesario que mantengas caracteres especiales deberías sanitizar la entrada (más de esto en la siguiente sección)
- Nunca utilices directamente una cadena como parte de un comando, esto evita ataques como inyección SQL y de String formatting (muy propio de C), antes valida y/o sanitiza

# Validación de números

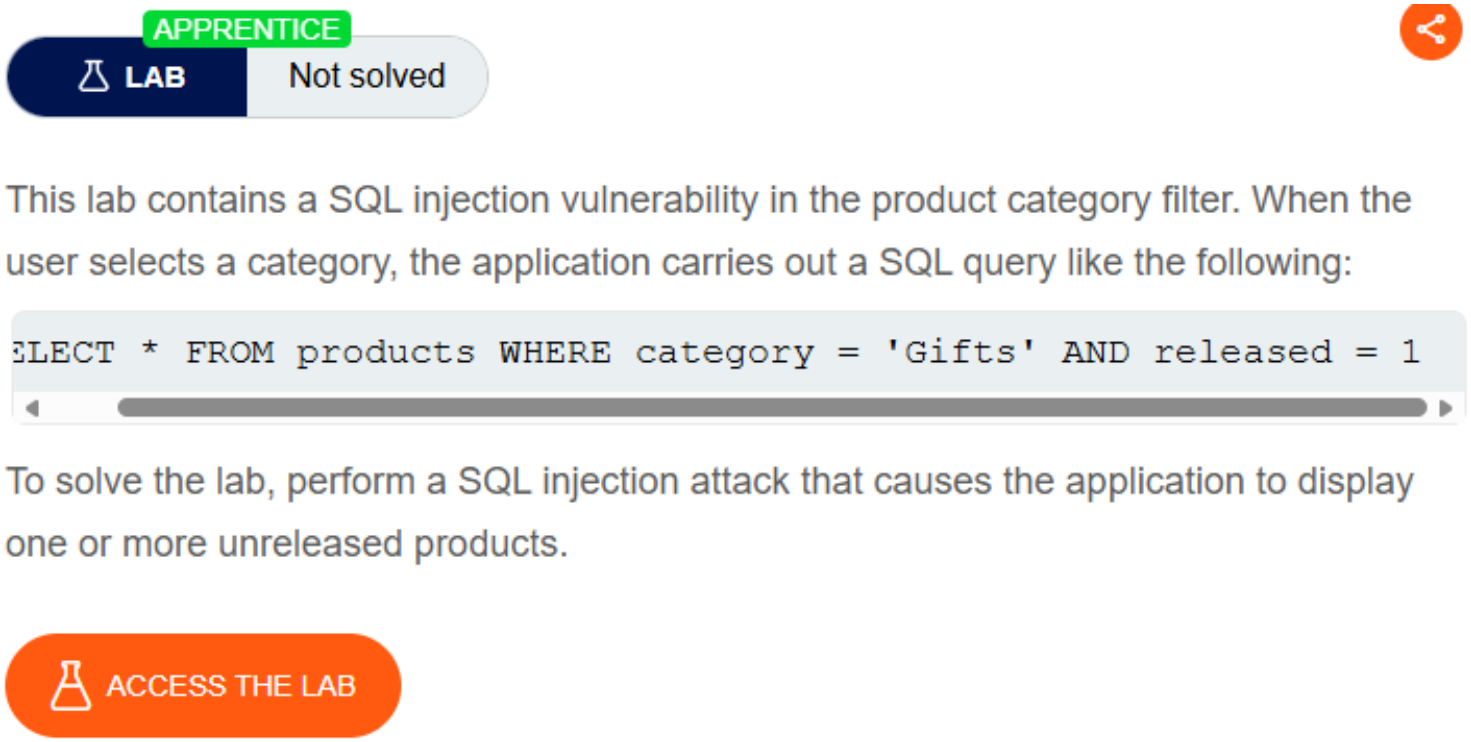


- Revisa que los valores que recibes sean realmente números del tipo que esperas (enteros o flotantes) antes de procesarlos, recuerda que en muchos contextos lo que recibes son en realidad cadenas que requieres convertir a número
- Revisa que el número que recibes esté en un rango mínimo y máximo
- Establece políticas adecuadas de redondeo de flotantes, por ejemplo para aplicaciones que manejen dinero (existen estándares para eso)
- Cuidado con las divisiones entre 0 u otros cálculos indeterminados (por ejemplo raíz cuadrada de un número negativo)

# Ejemplo: inyección SQL

## Laboratorio de SQL información escondida

- https://portswigger.net/web-security/sql-injection/lab-retrieve-hidden-data



# Sanitizar entradas



- Se refiere a limpiar entradas de usuario
- Esta limpieza se requiere para almacenar apropiadamente los datos o bien para limpiar posibles caracteres especiales que pueden ser usados en un ataque
- La limpieza más común de entrada consiste en quitar espacios en blanco y saltos de línea al principio y al final de una cadena

# Sanitizar entradas



- Lo anterior puede lograrse mediante la función de Java trim que pertenece a la case String
- En Python se logra mediante la función strip de algún objeto str

# Sanitizar entradas



- Para aplicaciones Web una sanitización tradicional consiste en remplazar caracteres especiales como < > ; “ ” por entidades HTML
- https://www.w3schools.com/html/html_entities.asp

# Prepared statements



- Muchos APIs de programación cuentan con una forma automatizada para aplicar sanitización
- La más común son los prepared statements (sentencias preparadas)
- La idea básica es no utilizar concatenaciones directas de cadenas con datos de entrada
- En su lugar se crean plantillas de comando donde las partes que tienen que ver con entradas se dejan sin especificar mediante un placeholder (como ? por ejemplo)

# Prepared statements



- Ejemplo: select * from usuarios where nombre = ? and edad
  - = ?
- Luego se pasa la plantilla a una función que realiza el remplazo llevando a cabo la sanitización adecuada

# Otros problemas de seguridad

# Manejo de memoria



- Estos consejos aplican principalmente a lenguajes donde el manejo de memoria es manual, como en el caso de C y C++
- Sin embargo, algunos consejos son genéricos para cualquier lenguaje de programación
- Los malos manejos de memoria pueden derivar en errores en tiempo de ejecución y en el peor de los casos de ejecución arbitraría de código

# Manejo de memoria



- Siempre valida que las entradas que recibes están en un rango de tamaño máximo, esto puede evitar un ataque de buffer overflow

  https://community.owasp.org/vulnerabilities/Buffer_Overflow

- Siempre valida los índices de control de acceso a estructuras indexadas como arreglos antes de intentar el acceso
- En caso de que aplique, siempre libera la memoria que ya no usas, esto evita que sufras una posible vulnerabilidad de memory lea

  https://community.owasp.org/vulnerabilities/Memory_leak

# Control de recursos



- Se refiere principalmente a archivos y conexiones (sockets)
- El consejo básico general es siempre cerrar el recurso después de haberlo usado
- No hacerlo puede provocar que se acaben los descriptores de archivo para un proceso o bien que se dejen puertos abiertos que puedan ser explotados

# Control de recursos



- Una forma general de asegurarse de que los recursos se cierren es utilizando un bloque finally dentro de un try-catch, así independientemente de algún fallo el recurso se cerrará
- También es importante manejar adecuadamente los errores que se pueden generar por un mal manejo de recursos (permisos, paths, puertos abiertos, etc.)